Lecture 4 — MapReduce & Databases

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

Overview Topics Covered

Part 1 · HDFS

The Hadoop Distributed File System — assumptions, design decisions, architecture (Namenode & Datanodes), read/write flows.

Part 2 · Relational Databases

Projection, Selection, Group-by / Aggregation — mapping SQL operations to MapReduce.

Part 3 · Joins

Broadcast (Map) Join vs. Reduce-side (Common) Join — trade-offs and use cases.

Supplementary: Secondary Sort — a technique to ensure values arrive at the reducer in sorted order, useful for quantiles, medians, and time-series data.

1 Why HDFS? The Problem with Separate Compute & Storage

In a naïve cluster, compute nodes and storage nodes are separate. This means every time a computation runs, data must be pulled from storage nodes across the network to the compute nodes. At big-data scale, this creates **enormous network traffic** and becomes the bottleneck.

HDFS solution: Co-locate storage and computation on the same machines. Each worker node stores data locally *and* runs computation on it. This is "moving the task to the data" — a core design principle from Lecture 2.

2 HDFS Assumptions

Commodity hardware

Designed to run on cheap, off-the-shelf machines — not expensive specialised servers. Scale out, not up.

High failure rates

Cheap components fail frequently. The system is designed to expect and tolerate failures, not avoid them.

Modest number of huge files

HDFS is optimised for a relatively small number of very large files (GBs to TBs each), not millions of tiny files.

Sequential reads

Workloads are read-heavy and sequential. Random access is not a goal. Sequential throughput is maximised.

3 HDFS Design Decisions

Files stored as fixed-size blocks

Each file is split into blocks of **128 MB** by default. Large block size reduces metadata overhead and encourages sequential access.

Reliability through replication

Each block is stored on **3 replicas** across different nodes by default. If one node fails, data is still available from another replica.

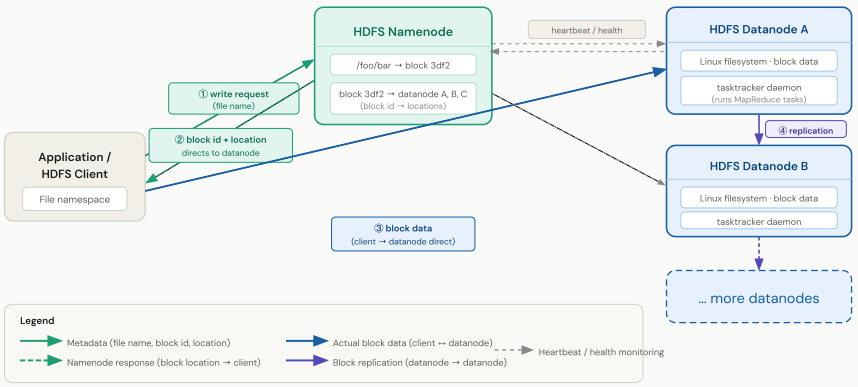
Single master (Namenode)

One master node coordinates all metadata. Keeps things simple and centralised, but is a single point of failure (mitigated by backups and checkpointing).

Write-once, read-many: HDFS does not support modifying files in place — only appending. This simplifies the consistency model but means HDFS is not suitable for workloads that require in-place updates.

4 HDFS Architecture: Namenode vs. Datanodes

ARCHITECTURE DIAGRAM — WRITING DATA (ADAPTED FROM SLIDE 8)



NAMENODE — Master · 1 per cluster

Manages file namespace

Stores directory structure, file-to-block mapping, access permissions, and block locations on each Datanode.

Coordinates access

Directs clients to the correct Datanode for reads and writes. **No actual data ever passes through the Namenode** — metadata only.

Health monitoring

Receives periodic "heartbeat" signals from every Datanode. If a heartbeat stops, the node is marked failed and its blocks are re-replicated elsewhere.

Block rebalancing

Ensures blocks are distributed roughly evenly across the cluster and that each block always has the required number of replicas (default: 3).

DATANODES — Workers · many per cluster

Stores actual data blocks

Each block (128 MB by default) is stored on the node's local Linux filesystem. One file is spread across many Datanodes.

Serves client requests

Clients communicate directly with Datanodes to read and write block data — the Namenode is not involved in data transfer.

Replicates blocks

When writing, the first Datanode pipes data to the next in the replica chain until all 3 replicas are stored on different machines.

Dual role in Hadoop

Each node also runs a *TaskTracker* (Hadoop) alongside the *Datanode daemon* (HDFS), enabling data-local computation.

HDFS CLIENT — The application reading or writing data

Asks the Namenode for block locations, then contacts Datanodes directly to transfer data. The client never moves data through the Namenode — only metadata requests go there.

Critical exam point: If the Namenode's data is lost without a backup, *all files on the filesystem become unrecoverable* — there is no way to reconstruct which blocks form which file from raw block data alone. Hadoop mitigates this with periodic backups and checkpointing (details out of scope).

5 HDFS: Writing & Reading Data

Writing Data

1. Client sends a **write request** to the Namenode.
2. Namenode checks the namespace, **allocates blocks**, and tells the client which Datanode (and replicas) to write to.
3. Client streams each block **directly to the first Datanode**.
4. That Datanode **pipes the block** to the next replica Datanode, which pipes to the next, until all 3 replicas are written.

Reading Data

1. Client sends a **read request** to the Namenode for a specific file.
2. Namenode looks up the file namespace and returns the location of the **closest Datanode** holding each block. "Closest" = same rack first, then same data centre.
3. Client contacts each Datanode **directly** to retrieve the block data. No data flows through the Namenode.

6 Putting It All Together: Hadoop + HDFS

The same physical machines serve dual roles simultaneously — as HDFS Datanodes (storage) and as Hadoop worker nodes (computation). This is the key to data locality.

Data locality: Hadoop's scheduler tries to assign each map task to run on the machine that already stores the needed input split. This avoids moving data across the network entirely — the computation goes to the data, not the other way around. This is one of MapReduce's biggest performance wins at scale.

Namenode = Hadoop Master

The same machine runs the HDFS Namenode and the Hadoop JobTracker (master scheduler for MapReduce jobs).

Datanodes = Hadoop Workers

Each worker node runs a Datanode daemon (stores HDFS blocks) and a TaskTracker (executes map/reduce tasks).

7 Relational Databases — Quick Refresher

A **relational database** is made up of tables. Each table represents a relation — a collection of tuples (rows). Each tuple consists of multiple fields (columns). SQL is the standard query language for relational databases.

Star Schema is a common design pattern for data warehouses: one central *fact table* (e.g. Sales) surrounded by multiple *dimension tables* (e.g. Product, Time, Customer). SQL queries often join these tables together and aggregate results.

8 Projection in MapReduce

Projection selects a subset of **columns** from a table (the SQL `SELECT col1, col2 FROM table` operation).

SQL equivalent

```
SELECT product_id, price FROM Sales
```

MapReduce implementation

Map: Take in a tuple (with tuple ID as key), emit a new tuple containing only the desired columns.

No reducer needed. This is a *map-only job* — the output of the map phase is directly the final output. There is no shuffle step, making it very efficient.

> Map-only jobs skip the shuffle entirely. They are the most efficient MapReduce jobs since there is no network transfer of intermediate data.

9 Selection in MapReduce

Selection filters **rows** based on a predicate (the SQL `WHERE` clause).

SQL equivalent

```
SELECT * FROM Sales WHERE price > 10
```

MapReduce implementation

Map: Take in each tuple. Emit the tuple only if it satisfies the predicate (e.g. price > 10). Discard the rest.

No reducer needed. Again, a map-only job.

10 Group By / Aggregation in MapReduce

Aggregation computes summary statistics over groups of rows — corresponds to `GROUP BY` + aggregate functions in SQL.

SQL equivalent — average sale price per product

```
SELECT product_id, AVG(price) FROM Sales GROUP BY product_id
```

MapReduce implementation

Map: Iterate over tuples; emit `(product_id, price)`.
Shuffle: Framework automatically groups all prices by product_id.
Reduce: For each product_id, receive a list of prices. Compute the average.

Note on combiners: You cannot use a combiner naively for AVG since it is not associative. Instead, emit `(product_id, (sum, count))` from the mapper; the combiner can then sum these pairs; the reducer computes the final average from summed (sum, count) pairs.

> **Combiner caution for AVG:** `mean(mean(3,5), 7) = mean(4, 7) = 5.5 ≠ mean(3,5,7) = 5`. Instead emit partial sums and counts to safely combine.

11 Relational Joins (Inner Join)

A join combines rows from two tables based on a matching key. For example, joining a *Citizenship* table (User_id, Country) with a *Modules* table (User_id, Module) on User_id.

One-to-One

Each key in table A matches at most one key in table B. Simplest case.

One-to-Many

Each key in table A matches multiple rows in table B (or vice versa).

Many-to-Many

A key can appear multiple times in both tables. The join example (users & modules) is many-to-many — a user can take many modules, and many users can take each module.

There are two main strategies for implementing joins in MapReduce:

12 Method 1: Broadcast Join (Map Join)

Requirement: One of the two tables must be small enough to fit entirely in memory on each worker node.

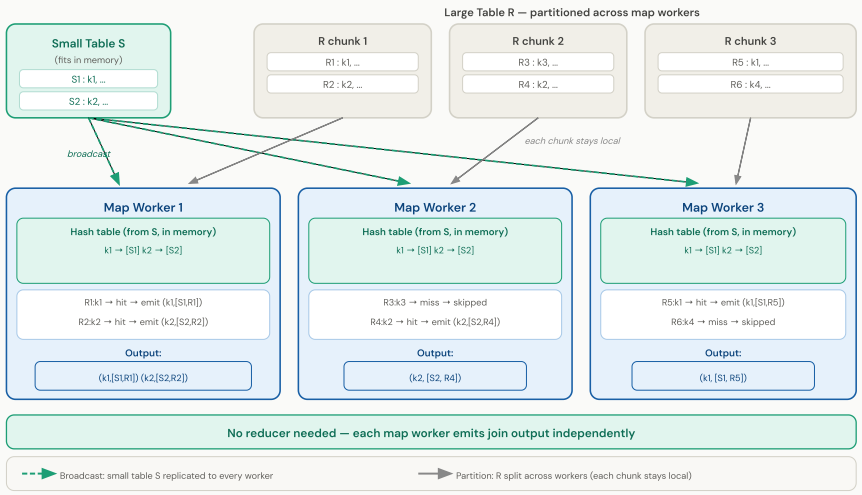
How it works

1. The **small table** is loaded into memory on every map worker as a hash table (keyed by the join attribute).

2. Each map worker then iterates over the **large table**, and for each row, looks up matching rows from the in-memory hash table to produce join output.

3. **No reducer needed** — all joining happens in the map phase. The shuffle step is completely avoided.

BROADCAST JOIN — HOW IT WORKS (S = SMALL TABLE, R = LARGE TABLE, 3 WORKERS)



Advantage

Extremely fast — no shuffle, no reducer. Network traffic is minimal since the small table is broadcast once and the large table is read locally.

Limitation

Requires the small table to fit in memory. If both tables are large, this approach is not feasible.

```
// Pseudocode: Broadcast Join
setup():    load small table S into hash map H (keyed by join_key)
map(key, row_from_R):
    matches = H.lookup(row_from_R.join_key)
    for each match in matches:
        emit(join_key, combine(row_from_R, match))
```

Worked Example — Citizenship ⋈ Modules on user_id

Table X — Citizenship **small table**

user_id	country
0	SG
1	MY

Table Y — Modules **big table**

user_id	module
0	CS4225
0	CS3230
1	CS4225

Table X (Citizenship) is small — it is broadcast to every mapper as a hash table. Table Y (Modules) is the big table that each mapper streams through row by row.

0. **Before mapping:** Table X is loaded into a hash table on *every mapper machine*, keyed by `user_id`. A hash table gives O(1) lookup per row.

```
Hash table loaded on every mapper (built from Table X):
  { 0: [SG],  1: [MY] }
```

1. **Map (Table Y only):** mappers iterate over Table Y. For each row, look up its `user_id` in the in-memory hash table and emit the joined result immediately — no coordination with other mappers needed.

```
Row (user_id=0, module=CS4225):  lookup 0 → [SG]  →  emit (SG, CS4225)
Row (user_id=0, module=CS3230):  lookup 0 → [SG]  →  emit (SG, CS3230)
Row (user_id=1, module=CS4225):  lookup 1 → [MY]  →  emit (MY, CS4225)
```

2. **No shuffle, no reducer.** The join is complete inside the map phase. Output goes directly to the final result:

```
Output: (SG, CS4225)   (SG, CS3230)   (MY, CS4225)
```

The core intuition: Broadcast join trades *memory* for *network I/O*. You use more memory (the entire small table lives on every mapper), but you eliminate the expensive shuffle step entirely. The hash table does the matching work that the shuffle would have done.

When this breaks down: If Table X were millions of rows, it would not fit in memory on every mapper. In that case, use reduce-side join instead — it has no memory constraint.

13 Method 2: Reduce-Side Join (Common Join)

When to use: When neither table fits in memory. Slower than broadcast join but works for any table size.

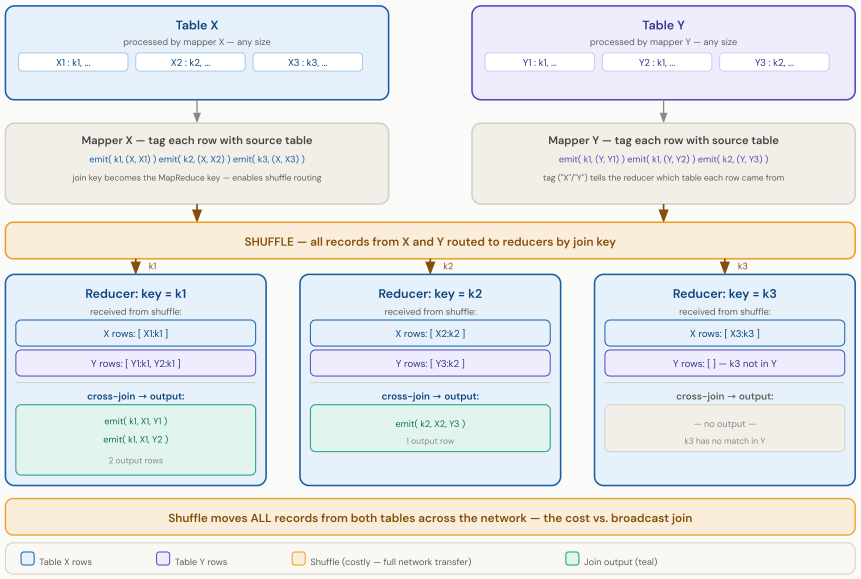
How it works

1. **Map phase:** Different mappers operate on each table. Each mapper emits records with the **join key** as the key, and a tagged value indicating which table the record came from: e.g. `(join_key, (X, record))` for table X, `(join_key, (Y, record))` for table Y.

2. **Shuffle:** All records with the same join key are sent to the same reducer (from both tables).

3. **Reduce phase:** The reducer receives all records for a given key from both tables. It cross-joins them to produce the output. Secondary sort (see Supplementary) can be used to ensure records from one table arrive before the other, making the join more memory-efficient.

REDUCE-SIDE JOIN — THREE PHASES: MAP · SHUFFLE · REDUCE



Advantage

Works for tables of any size — no memory constraint. Scales to arbitrarily large datasets.

Limitation

Slower than broadcast join due to shuffle overhead — all records from both tables must be transferred across the network.

```
// Pseudocode: Reduce-side Join
map_X(key, row):  emit(row.join_key,  ("X", row))
map_Y(key, row):  emit(row.join_key,  ("Y", row))

reduce(join_key, values):
    x_rows = [v for v in values if v.tag == "X"]
    y_rows = [v for v in values if v.tag == "Y"]
    for x in x_rows:
        for y in y_rows:
            emit(join_key, combine(x, y))
```

Worked Example — Citizenship ⋊ Modules on user_id

Table X — Citizenship

user_id	country
0	SG
1	MY

Table Y — Modules

user_id	module
0	CS4225
0	CS3230
1	CS4225

Goal: join on `user_id` to produce `(country, module)` pairs. Neither table needs to fit in memory.

1. **Map (both tables separately):** use `user_id` as the MapReduce key. Tag each emitted record with its source table so the reducer can tell them apart.

```
From Table X:
  emit( key=0, tag=X, value=SG )
  emit( key=1, tag=X, value=MY )
From Table Y:
  emit( key=0, tag=Y, value=CS4225 )
  emit( key=0, tag=Y, value=CS3230 )
  emit( key=1, tag=Y, value=CS4225 )
```

2. **Shuffle:** the framework groups all records by `user_id`. All rows sharing the same join key — from both Table X and Table Y — land at the same reducer. This is the key mechanic that makes the join work.

```
key=0  →  [ (X, SG),  (Y, CS4225),  (Y, CS3230) ]
key=1  →  [ (X, MY),  (Y, CS4225) ]
```

3. **Reduce:** secondary sort ensures all X rows arrive *before* Y rows. The reducer holds X rows in memory, then streams through Y rows and crosses each one with every stored X row.

```
Reducer for key=0:
  X in memory: [ SG ]
  SG × CS4225  →  emit (SG, CS4225)
  SG × CS3230  →  emit (SG, CS3230)

Reducer for key=1:
  X in memory: [ MY ]
  MY × CS4225  →  emit (MY, CS4225)
```

Why secondary sort? The reducer must read all X rows before it can start crossing with Y rows. Secondary sort guarantees X records arrive first — so the reducer only needs to buffer X rows in memory. Y rows are then streamed through one at a time, keeping memory usage bounded.

The core intuition: The shuffle acts as the "matching" step of the join — it automatically brings all rows sharing the same `user_id` (from both tables) to the same reducer. The reducer then just needs to cross-pair them. The cost is that *all* records from both tables travel across the network during the shuffle.

14 Join Comparison Summary

Property	Broadcast Join (Map Join)	Reduce-side Join (Common Join)
Requirement	One table must fit in memory	No memory constraint
Shuffle needed?	No — map-only job	Yes — full shuffle of both tables
Speed	Faster (no network overhead)	Slower (full network transfer)
Scalability	Limited by memory per worker	Scales to any dataset size
Best for	One large + one small table	Two large tables

15 Exam-Style Q&A

Q: What happens if the Namenode fails and its data is lost?

> All files on the filesystem cannot be retrieved — there is no way to reconstruct files from raw block data alone, since the Namenode holds the only mapping from file names to blocks. Hadoop mitigates this with backups and checkpointing.

Q: Does data pass through the Namenode during reads and writes?

> No. The Namenode only provides metadata (block locations). The actual data is transferred directly between the client and the Datanodes.

Q: Why is HDFS block size set to 128 MB by default?

> Large blocks reduce metadata overhead (fewer blocks to track in the Namenode) and encourage sequential access patterns. Too small = too many blocks, overwhelming the Namenode. Too large = less parallelism in MapReduce.

Q: Which relational operations require a reducer in MapReduce?

> Projection and Selection are map-only (no reducer needed). Group-by Aggregation and Joins require reducers (aggregation happens in the reduce phase).

Q: When should you use Broadcast Join vs. Reduce-side Join?

> Use Broadcast Join when one table is small enough to fit in memory on each worker — it is faster (no shuffle). Use Reduce-side Join when both tables are large and neither can fit in memory — it is slower but works at any scale.

Q: In Secondary Sort, why must you implement a custom Partitioner?

> Because the key is now a composite (K1, K2). The default hash partitioner would partition by the full composite key, potentially sending (Jan, 20°C) and (Jan, 30°C) to different reducers. The custom partitioner hashes by K1 only, ensuring all data for the same month goes to the same reducer.

Q: How does data locality improve MapReduce performance?

> Hadoop tries to schedule each map task on the machine that already stores the required input split. This avoids transferring data over the network entirely — the task runs where the data is. This dramatically reduces network traffic and speeds up jobs.

SUPPLEMENTARY

SUPPLEMENTARY Secondary Sort

Problem: In MapReduce, keys arrive at the reducer in sorted order, but *values for each key arrive in an arbitrary (unsorted) order*. If you need values to arrive sorted (e.g. temperatures sorted within each month, for computing medians and quantiles), you need Secondary Sort.

Motivating example

Mappers emit `(Jan, 30°C)` and `(Jan, 20°C)`. The reducer receives `Jan: [30°C, 20°C]` — order is arbitrary. We want `Jan: [20°C, 30°C]` (sorted) so we can efficiently find the median or quantiles.

Secondary Sort approach — 3 steps:

1. **Create a composite key** = (natural key K1, sort key K2). E.g. instead of key `Jan`, use composite key `(Jan, 20°C)`. Define a custom comparator that sorts by K1 first, then by K2.
2. **Custom Partitioner:** Partition by K1 only (not the full composite key). Without this, `(Jan, 20°C)` and `(Jan, 30°C)` could be hashed to different reducers, splitting January data across multiple reducers — incorrect!
3. **Result:** All records for the same K1 go to the same reducer (due to custom partitioner), and within that reducer, records arrive sorted by K2 (because the framework sorts by the full composite key). You get sorted values for free.

Why not just sort in the reducer? You can, but it may be very expensive in memory and time if there are many values for one key. Secondary Sort "borrows" Hadoop's built-in distributed sorting mechanism, which is already highly optimised, to sort the values without loading them all into memory at once.

Secondary Sort Summary Table

Component	What to customise	Purpose
Key	Use composite key (K1, K2)	Enables sorting by K2 within groups of K1
Comparator	Sort by K1 first, then K2	Determines the order records arrive at the reducer
Partitioner	Partition by K1 only	Ensures all records with the same K1 go to the same reducer